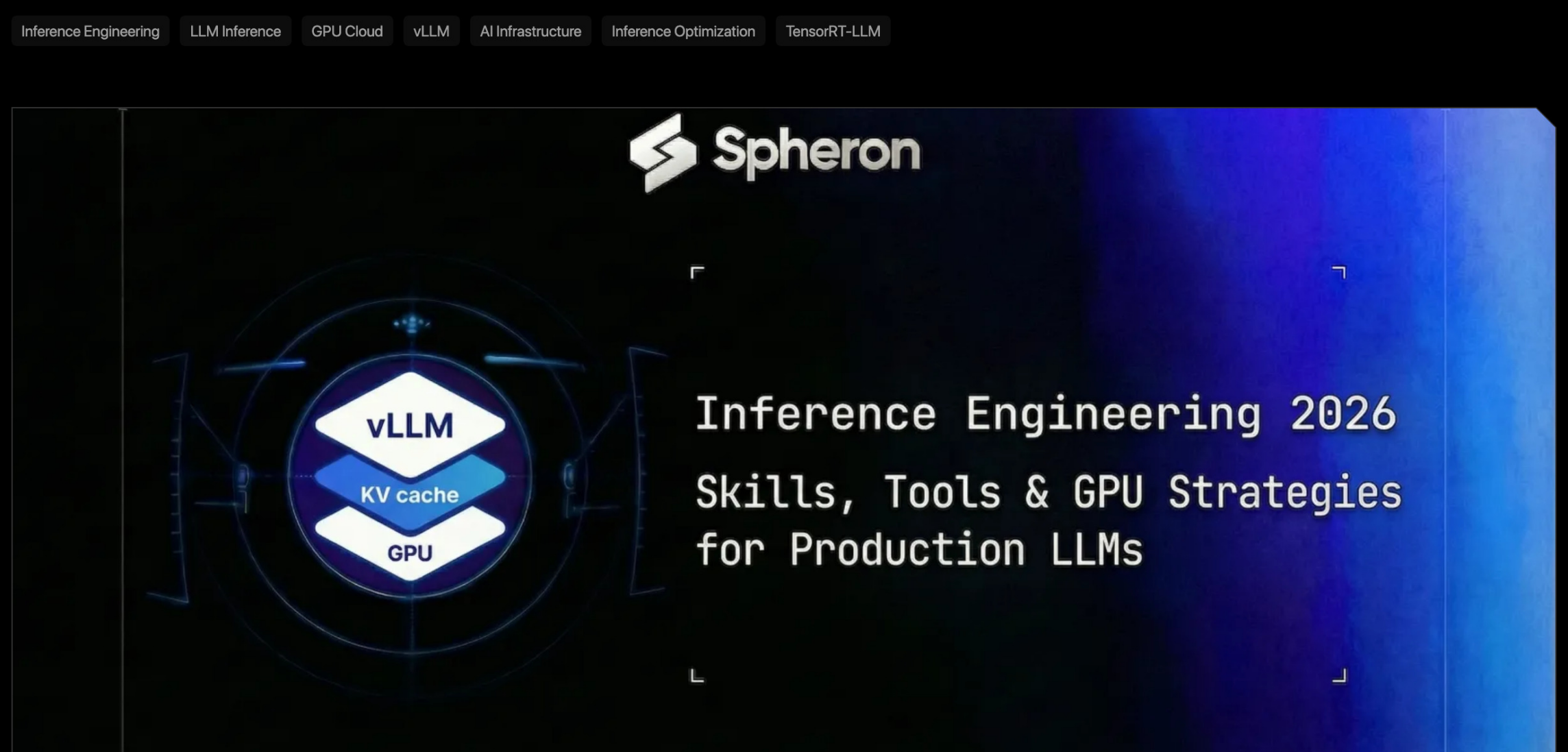
ENGINEERING

What Is Inference Engineering? The 2026 GPU Cloud Guide

← BACK TO BLOG • WRITTEN BY MITRASISH, CO-FOUNDER & CTO, SPHERON • PUBLISHED APR 7, 2026 | UPDATED OCT 3, 2026

Inference Engineering · LLM Inference · GPU Cloud · vLLM · AI Infrastructure · Inference Optimization · TensorRT-LLM


Spheron
vLLM
KV cache
GPU
Inference Engineering 2026
Skills, Tools & GPU Strategies for Production LLMs

Inference now accounts for the majority of AI compute in production systems. Training ends when the model ships. Inference runs every hour, every day, as long as users are hitting your API. The discipline that makes that sustainable at scale is inference engineering.

Inference engineers own four things: hardware selection, serving framework configuration, cost-per-token optimization, and reliability SLAs. If you are deploying a model to production and care about whether it serves 10 users or 10,000 without tripling your GPU bill, inference engineering is the work.

What Is Inference Engineering

Inference engineering is the practice of serving trained machine learning models to users at scale. It is not about building models. It is about making them fast, affordable, and reliable after they are built.

The discipline emerged as LLMs went to production. A 70B parameter model running on a single GPU naively processes one request at a time. An inference engineer gets that same model to serve hundreds of concurrent users at a fraction of the cost per token, using batching, quantization, caching, and hardware selection.

What inference engineers own:

- **Hardware selection**: which GPU, how many, which memory tier
- **Serving framework configuration**: vLLM, SGLang, TensorRT-LLM, or NVIDIA Dynamo
- **KV cache management**: memory allocation, prefix caching, quantization
- **Cost-per-token optimization**: throughput vs latency trade-offs, spot vs on-demand, right-sizing
- **Reliability SLAs**: time-to-first-token targets, inter-token latency budgets, uptime guarantees. For a complete framework on setting TTFT and ITL SLOs, including error budgets, capacity math, and autoscaling triggers, see the LLM inference SLO engineering guide.

The application-level counterpart to these server-side concerns is deciding what tokens go into context to minimize prefill cost. That discipline is covered in the context engineering for AI agents guide.

Inference Engineering vs ML Engineering vs MLOps

These three roles share territory but own different things. The confusion is common because small teams often combine them.

Discipline	Owns	Does Not Own
ML Engineering	Model training, datasets, evaluation, fine-tuning pipelines	Serving infrastructure, hardware costs, latency SLAs
MLOps	CI/CD for ML, experiment tracking, model registry, pipeline orchestration	Serving optimization, GPU memory management, cost-per-token
Inference Engineering	Serving frameworks, GPU selection, KV cache tuning, throughput/latency trade-offs, inference FinOps	Model architecture, training code, data pipelines

The split happened because serving LLMs at scale requires depth that generalist ML and DevOps roles do not cover. You need to understand GPU memory hierarchies, kernel-level batching behavior, quantization accuracy trade-offs, and cost-per-token math simultaneously. That depth is its own specialty.

Core Skills Every Inference Engineer Needs

Batching and Scheduling

Static batching groups requests into fixed batches and waits for all of them to finish before accepting new ones. If one request in a batch generates 2,000 tokens and another generates 20, every GPU cycle spent waiting for the long request is wasted capacity.

Continuous batching fixes this by operating at the iteration level. At each decode step, finished requests are immediately replaced by new ones from the queue. The GPU stays full. Throughput goes up 3-5x compared to static batching for typical LLM workloads.

For agentic inference specifically, batching and GPU scheduling are only half the picture. Tool calls, tokenization threads, and orchestration overhead run on CPU between inference steps. Optimizing vCPU per GPU for agentic inference matters as much as batching configuration once concurrent agent sessions exceed a few dozen.

For a deep technical breakdown of how continuous batching, PagedAttention, and chunked prefill work together, see our LLM serving optimization guide. If you are new to the inference engineering discipline, read that post alongside this one for the full picture before diving into framework configuration.

KV Cache Management

The KV cache stores key-value attention tensors for every token the model has processed. It is what allows the model to generate subsequent tokens without reprocessing the entire context from scratch.

The problem is size. For a 70B model with FP16 weights, the KV cache for a single 4,096-token request takes roughly 4GB of VRAM. At 50 concurrent requests, that is 200GB, more than any single GPU holds. KV cache management is how you fit more concurrent requests into available VRAM.

Key techniques:

- **PagedAttention**: allocates KV cache in fixed-size blocks rather than contiguous chunks, eliminating fragmentation and wasted pre-allocation
- **Prefix caching**: reuses KV cache for identical system prompts across requests, reducing compute for shared-prompt workloads by 20-40%
- **KV cache quantization**: storing KV tensors in INT8 or INT4 instead of FP16 cuts memory requirements by 2-4x at modest accuracy cost

For a deeper look at each of these techniques and how to configure them in vLLM, see our KV cache optimization guide. Semantic caching at the application layer can yield 30-70% hit rates on agent and FAQ traffic - see the semantic cache deployment guide for setup details.

Quantization

Quantization reduces the numerical precision of model weights and activations, which shrinks memory footprint and increases throughput.

The practical progression:

Precision	Memory vs FP16	Throughput Gain	Accuracy Impact
FP16	1x (baseline)	Baseline	None
FP8	0.5x	1.5-2x	Under 1% on most models
INT4 AWQ	0.25x	2-3x	1-3% on sensitive tasks

FP8 is the default for H100 and B200 hardware. It halves memory bandwidth requirements with negligible accuracy loss on Llama 3, Qwen, and DeepSeek class models. INT4 AWQ works well for A100-class GPUs where VRAM is the binding constraint.

Serving Frameworks

Choosing the right serving framework has more impact on cost and throughput than any hardware upgrade. The framework controls batching behavior, memory management, kernel selection, and API compatibility. The next section covers the four main options in 2026.

The Inference Stack in 2026

vLLM

vLLM introduced PagedAttention in 2023 and became the default open-source LLM serving framework. Its V1 engine (default since 2025, now at v0.19.0) adds chunked prefill, improved scheduler logic, and better multi-modal support.

Best for: general LLM serving, teams starting out, OpenAI API drop-in replacement, any workload where ecosystem compatibility matters more than maximum raw throughput.

Strengths: widest model support, active community, OpenAI-compatible REST API, native FP8 quantization on H100.

Trade-offs: not the fastest option for throughput-critical deployments; TensorRT-LLM beats it on raw NVIDIA hardware performance.

For production deployment configuration, see our vLLM production deployment guide.

SGLang

SGLang uses RadixAttention, a variant of prefix caching that automatically identifies and reuses shared prefixes across requests in the batch. For multi-turn conversations and agentic workloads with repeated system prompts, this makes a measurable difference.

Best for: agentic workflows with tool calls and multi-turn context, structured output generation (JSON schema enforcement), workloads with high prefix reuse across requests.

Strengths: RadixAttention prefix caching, constrained generation for structured outputs, lower TTFT on repeated-prefix workloads.

Trade-offs: smaller ecosystem than vLLM; fewer model integrations and less community documentation for edge cases.

NVIDIA Dynamo

NVIDIA Dynamo is designed for disaggregated serving across multiple GPU nodes. It handles KV cache routing between prefill and decode workers, with NIXL (NVIDIA Inference Xfer Library) managing tensor transfer between nodes.

Best for: teams running 70B+ models at high load across multiple GPUs, disaggregated prefill-decode deployments, multi-node inference at scale.

Strengths: purpose-built for multi-node disaggregation, native NIXL integration, supports heterogeneous GPU pools for prefill vs decode.

Trade-offs: operational complexity; overkill for single-node deployments. For the full disaggregated serving architecture, see our prefill-decode disaggregation guide.

TensorRT-LLM

TensorRT-LLM compiles models into optimized CUDA engines using NVIDIA's TensorRT runtime. The compilation step is time-consuming (30-90 minutes per model per GPU type), but the resulting engines deliver the highest raw throughput on NVIDIA hardware.

Best for: latency-critical production deployments on NVIDIA hardware where engineering time is available for engine compilation and maintenance.

Strengths: highest throughput on NVIDIA GPUs, FP4 support on Blackwell (B200), in-flight batching with custom CUDA kernels.

Trade-offs: compile-time overhead, harder to update models quickly, NVIDIA-only, steeper learning curve than vLLM.

For a side-by-side comparison, see our vLLM vs TensorRT-LLM vs SGLang benchmarks.

Framework decision matrix:

Framework	Best For	Strengths	Trade-offs
vLLM	General LLM serving, OpenAI API compatibility	Wide model support, easy setup	Not the fastest on raw throughput
SGLang	Multi-turn, agentic, structured outputs	RadixAttention, prefix reuse	Smaller ecosystem
NVIDIA Dynamo	Multi-node disaggregated serving	Scales across GPU pools	Complex ops, overkill for single-node
TensorRT-LLM	Latency-critical NVIDIA deployments	Highest raw throughput	Compilation overhead, NVIDIA-only

GPU Selection for Inference Workloads

GPU choice is the biggest cost lever in inference. Pick the wrong tier and you either overpay for capacity you cannot use (H200 on a 7B model) or cap throughput by running out of memory bandwidth (A100 on a 70B at high concurrency). The gap between optimal and suboptimal GPU selection is typically 2-4x in cost.

The Decision Matrix

Current pricing on Spheron:

GPU	HBM	Memory BW	Best For	On-Demand $/hr	Spot $/hr
A100 80GB SXM4	80 GB	2 TB/s	7B-34B models, cost-sensitive workloads	$1.43/hr	$1.19/hr
H100 SXM5	80 GB	3.35 TB/s	34B-70B high-throughput production	$2.65/hr	$2.20/hr
H200 SXM5	141 GB	4.8 TB/s	70B-405B memory-bound models	$4.80/hr	$3.35/hr
B200 SXM6	192 GB	8 TB/s	405B+, next-gen scale	$11.06/hr	$5.22/hr

> Pricing fluctuates based on GPU availability. Spheron rates above are live as of **16 Oct 2026**; other providers reflect their most recent published rates and may have changed. Check current GPU pricing → for live rates.

How to read this table:

A100 80GB SXM4 at $1.43/hr is the right choice for 7B-34B models at moderate concurrency, teams prioritizing cost over raw throughput, and batch inference jobs that can run on spot at $1.19/hr. Explore A100 GPU rental → for availability.

H100 SXM5 at $2.65/hr delivers the best price-performance for production serving of 34B-70B models. The 3.35 TB/s memory bandwidth handles high-concurrency decode without becoming the bottleneck. At $2.65/hr versus the A100's $1.43/hr, the H100 premium pays off when throughput matters: more memory bandwidth means more concurrent requests served per hour at 70B scale. For most production 70B deployments with FP8 quantization, H100 is the right default. See H100 GPU rental →.

H200 SXM5 at $4.80/hr is justified when model weights plus KV cache exceed 80GB. A 70B model in FP16 takes 140GB; in FP8 it takes 70GB. With FP8 quantization, most 70B models fit on H100. Move to H200 when you are running FP16, have long context windows that balloon KV cache, or are serving 405B-class models. See H200 GPU rental →.

B200 SXM6 at $11.06/hr is the right choice for 405B+ models and next-generation scale. The 192GB HBM and 8 TB/s bandwidth make it uniquely suited for the largest models. At $11.06/hr it is the most expensive option in this table, but for workloads that require 192GB of memory or maximum memory bandwidth, there is no comparable alternative. Spot pricing is available at $5.22/hr for batch workloads. See B200 GPU rental →.

Inference Cost Economics: Token-Level GPU FinOps

The metric that matters for inference budgeting is cost per million tokens (CPM). It converts GPU rental rates and throughput into a single comparable figure.

The formula:

```
CPM = (GPU $/hr) / (tokens/sec × 3600) × 1,000,000
```

Example: H100 SXM5 at $2.90/hr (Spheron on-demand rate as of April 2026) serving Llama 3 70B FP8 at 1,200 tokens/sec:

```
CPM = $2.90 / (1,200 × 3,600) × 1,000,000
    = $2.90 / 4,320,000 × 1,000,000
    = ~$0.67 per million tokens
```

For comparison, managed inference APIs for 70B-class models typically charge $0.50-$0.90 per million tokens. Self-hosted on Spheron at optimized configuration you can reach $0.20-$0.35/M tokens, with further reductions using spot instances.

For the full FinOps playbook with case studies and monthly budget optimization, see our AI inference cost economics guide. For a GPU-by-GPU breakdown of tokens per watt and CPM calculations across Hopper and Blackwell hardware, see the token factory economics guide.

The main cost levers, ranked by impact and implementation effort:

Lever	Typical Savings	Complexity
Continuous batching	2-3x throughput at same GPU cost	Low (default in vLLM/SGLang)
FP8 quantization	40-50% fewer tokens per dollar	Low (single flag in vLLM)
Right-sizing GPU	30-50% vs defaulting to H100 for small models	Low (one-time benchmark)
Spot instances	40-60% cost reduction	Medium (retry logic required)
Prefix caching	20-40% reduction for shared-prompt workloads	Medium (vLLM config)

Advanced Techniques

Speculative Decoding

Speculative decoding uses a small draft model to propose multiple tokens simultaneously, which the main model then verifies in a single forward pass. When the draft model's predictions match, you effectively generate 3-5 tokens per main model forward pass instead of one.

The catch: it only helps when the acceptance rate is above 80%. Below that, you are spending compute on rejected proposals and gaining nothing. Acceptance rates are highest for predictable, repetitive outputs: chat completions, code generation with clear patterns, document filling. They are lower for creative tasks with high output entropy.

Disaggregated Prefill-Decode Serving

Prefill (processing the input prompt) is compute-bound. Decode (generating tokens) is memory-bandwidth-bound. Running both phases on the same GPU means each phase interferes with the other.

Disaggregated serving splits them onto separate GPU pools: compute-dense GPUs handle prefill, memory-bandwidth-dense GPUs handle decode. For long-context workloads with 8K+ token prompts and high concurrency, this can deliver 60-75% throughput improvement over colocated serving.

This is an advanced inference engineering technique. The full setup guide is at prefill-decode disaggregation on GPU cloud.

Multi-Token Prediction

Models trained with multi-token prediction (MTP) heads can generate 2-4 tokens per forward pass without speculative decoding overhead. DeepSeek V3 includes MTP heads; other models trained with multi-token prediction objectives can also benefit. When running these models on compatible serving stacks, enable MTP to reduce wall-clock latency at no additional compute cost. Our MTP deployment guide walks through the vLLM and SGLang config and the specific 2-3x speedups we measured on DeepSeek V4 and GLM-5.1.

Tensor Parallelism and Pipeline Parallelism

Tensor parallelism (TP) splits individual model layers across multiple GPUs within a node. It is required for 70B models on H100 (70B FP16 = 140GB, does not fit in 80GB). Use TP=4 for 70B FP8, TP=8 for 70B FP16, TP=8 for 405B FP8.

Pipeline parallelism (PP) chains model stages across nodes. It adds inter-node communication latency to each forward pass but enables 405B+ models on GPU clusters with modest per-node counts. Use PP only when the model does not fit on a single node with tensor parallelism.

For H100 8-GPU nodes: start with TP=8 for 70B FP16 and TP=4 for 70B FP8. Verify near-linear throughput scaling before adding more nodes.

For teams running bare-metal PyTorch inference rather than vLLM or SGLang, `torch.compile` with persistent Inductor cache is a meaningful optimization. The torch.compile production inference guide covers CUDA graph capture, Inductor backend tuning, and how to persist compiled kernel artifacts to NVMe across restarts. For inference engineers who need custom fused operators beyond what the framework provides, you can write custom Triton kernels in Python and integrate them into vLLM, SGLang, or torch.compile without touching CUDA C++.

Building an Inference Engineering Practice on GPU Cloud

Here is a practical first-week setup for inference engineering on Spheron, structured as a sequence of progressively more optimized deployments.

Step 1: Baseline measurement

Rent two H100 SXM5s (70B FP16 requires ~140 GB VRAM, which exceeds a single 80 GB GPU). Run vLLM with default settings against your target model. Measure throughput (tokens/sec) and TTFT at 10, 50, and 100 concurrent users using `benchmark_serving.py`. This is your baseline. Every subsequent change gets measured against it.

```bash
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --dtype float16 \
  --tensor-parallel-size 2 \
  --max-model-len 8192
```

Step 2: Apply FP8 and tune KV cache

Enable FP8 quantization and raise `gpu-memory-utilization` to 0.95. Measure again at the same concurrency levels.

```bash
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --dtype float16 \
  --quantization fp8 \
  --tensor-parallel-size 2 \
  --gpu-memory-utilization 0.95 \
  --enable-chunked-prefill \
  --max-num-batched-tokens 8192
```

FP8 frees up VRAM for more KV cache, which lets vLLM hold more concurrent requests in flight. At 100 concurrent users, this step typically increases throughput 60-90% on 70B models.

Step 3: Scale with tensor parallelism

Move to a multi-GPU configuration with TP=4 or TP=8. Validate that throughput scales near-linearly. If it does not, the bottleneck has shifted to inter-GPU communication or scheduler overhead, not GPU compute.

Step 4: Optimize cost

Switch batch inference jobs (embeddings, async summarization, document processing) to spot instances. Add prefix caching for any workload where many users share the same system prompt. Calculate your actual CPM and compare against your per-user budget.

Spheron's GPU fleet includes H100, H200, B200, and A100 across data center partners globally, with per-second billing, bare metal for custom serving stacks, and pre-configured vLLM templates. For deployment configuration details, see the Spheron documentation.

For teams shipping LLM features in 2026, evaluation has become as critical as serving. See the LLM-as-judge evaluation pipeline guide for a production setup covering judge model selection, bias mitigation, and CI integration.

> Inference engineering teams need flexible GPU access, for benchmarking new models, testing quantization configs, and scaling production serving without being locked into reserved instances. Spheron gives you on-demand access to H100, H200, B200, and A100 GPUs with per-second billing, bare metal options for custom serving stacks, and pre-configured vLLM templates.
>
> Check H100 availability → | H200 GPU pricing → | Spheron B200 → | View all pricing →
>
> Get started on Spheron →

STEPS / 05

Quick Setup Guide

01 **Choose a serving framework**

Select vLLM for general LLM serving, SGLang for multi-turn and agentic workloads, TensorRT-LLM for maximum throughput on NVIDIA hardware, or NVIDIA Dynamo for disaggregated multi-node deployments.

02 **Select the right GPU tier**

Match GPU to workload: A100 80GB for models up to 70B at moderate load, H100 SXM5 for high-throughput production serving, H200 for memory-bound 405B+ models, B200 for next-generation scale.

03 **Configure KV cache and batching**

Set gpu-memory-utilization to 0.90, enable chunked prefill, tune max-num-seqs based on your concurrency target, and monitor KV cache hit rate to validate prefix caching is working.

04 **Apply quantization**

Run FP8 quantization as the default starting point on H100 and B200 hardware. It halves memory bandwidth requirements with under 1% accuracy loss on most models. Use INT4 AWQ for further compression on smaller GPUs.

05 **Benchmark and iterate**

Measure tokens-per-second throughput, time-to-first-token (TTFT), inter-token latency (ITL), and cost-per-million-tokens. Use Prometheus + Grafana with vLLM's built-in metrics endpoint to track production health.

FAQ / 05

Frequently Asked Questions

01 **What is inference engineering?** ×

Inference engineering is the discipline focused on serving trained ML models to users at scale, optimizing for throughput, latency, cost, and reliability. It sits at the intersection of ML, systems engineering, and FinOps.

02 **How is inference engineering different from ML engineering?** +

03 **What frameworks do inference engineers use?** +

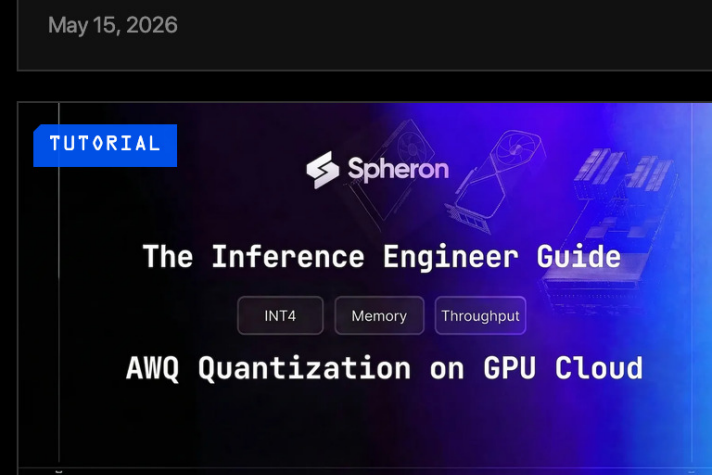
04 **Which GPU is best for LLM inference in 2026?** +

05 **How do you reduce inference costs?** +

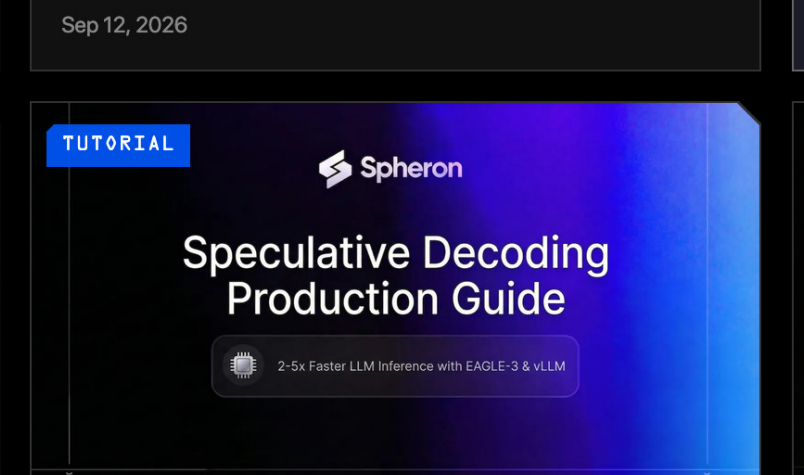
← Back to all posts

Related Reading

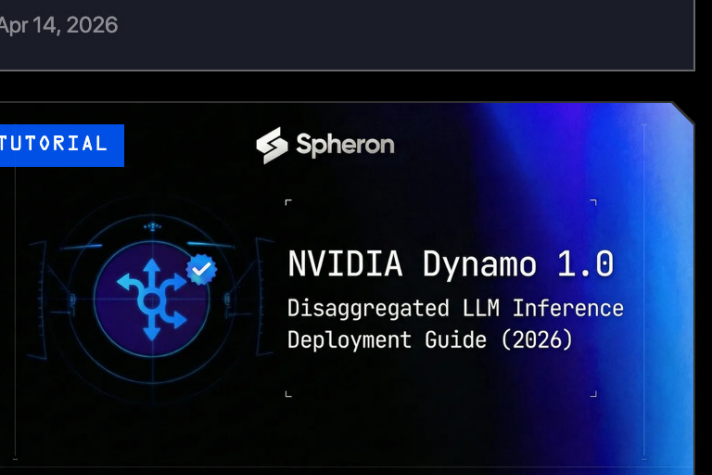
MORE ON INFERENCE OPTIMIZATION →


PILLAR GUIDE
Spheron
How Much VRAM Do You Need for LLMs?
7B to 70B+ Models • FP16, INT8, INT4

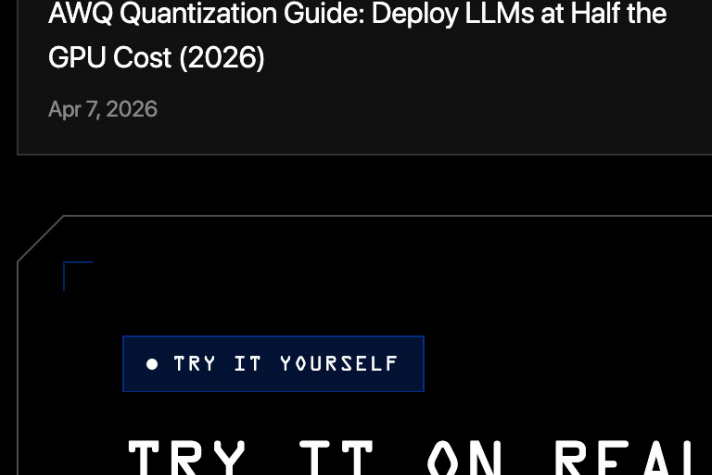
LLM VRAM Requirements: How Much GPU Memory You Need (2026)
May 15, 2026



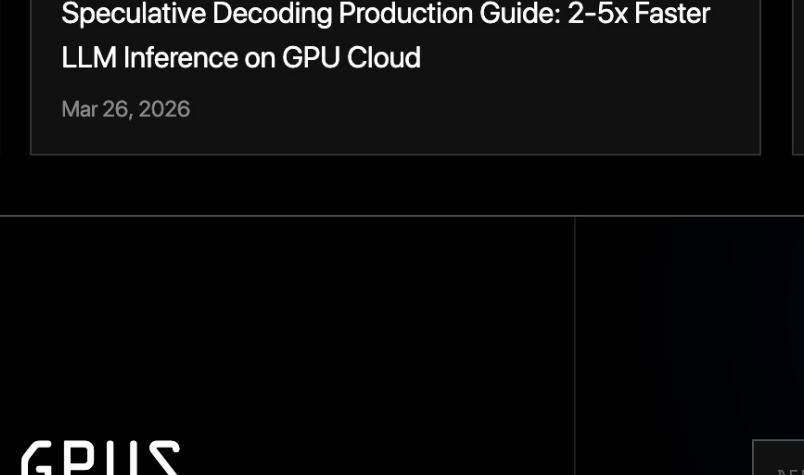
GPU Inference Throughput Scaling: The Batch Size Myth
Sep 12, 2026



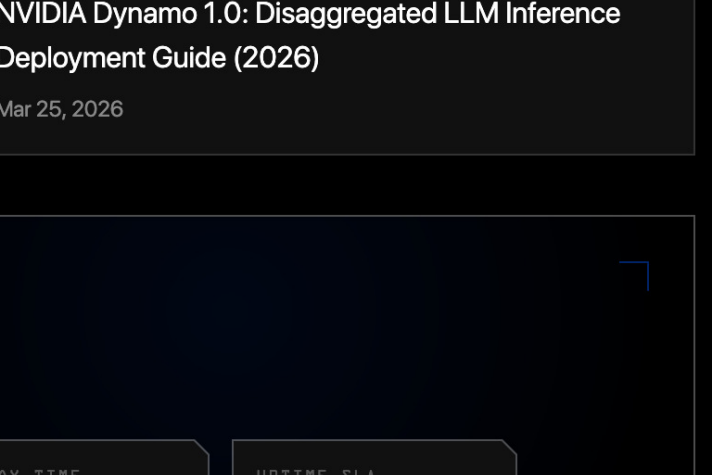
MXFP4 Quantization on GPU Cloud: Deploy LLMs at 4-Bit Precision (2026)
Apr 14, 2026



AWQ Quantization Guide: Deploy LLMs at Half the GPU Cost (2026)
Apr 7, 2026


TUTORIAL
Spheron
Speculative Decoding Production Guide
2-5x Faster LLM Inference with EAGLE-3 & vLLM

Speculative Decoding Production Guide: 2-5x Faster LLM Inference on GPU Cloud
Mar 26, 2026



NVIDIA Dynamo 1.0: Disaggregated LLM Inference Deployment Guide (2026)
Mar 25, 2026

• TRY IT YOURSELF

TRY IT ON REAL GPUS

The GPUs behind these guides are the ones you can rent here: H100s, H200s, B200s, and more, billed per minute after a 20-minute minimum runtime, with no contracts. Pick one and you are live in under two minutes.

Rent a GPU

DEPLOY TIME < 2 min
UPTIME SLA 99.9%
GPU MODELS 10+
STARTING AT $0.48/hr

GLOBAL COMPUTE, BROUGHT TO YOU BY
Spheron
Bare-metal GPU infrastructure for AI teams that need scale, speed, and simplicity.

Deploy Now
Contact Sale →

PRODUCT
App
Pricing
Customers
GPU Catalog
Supply GPU

RESOURCES
Documentation
API Reference
Blog
GitHub
Changelog

TOOLS
GPU Recommender
Training Cost Calculator

FEATURES
On-Demand Instances
Spot Instances
Reserved Commitments

COMMUNITY
Discord
X (Twitter)
LinkedIn
YouTube
Telegram

GPUS
NVIDIA Vera Rubin
NVIDIA GB300
NVIDIA GB200
NVIDIA B300
NVIDIA H100
NVIDIA B200
NVIDIA H200
NVIDIA A100
NVIDIA GH200
NVIDIA L40S
AMD MI300X
RTX 4090
RTX 5090
RTX PRO 6000

© 2026 SPHERON. ALL RIGHTS RESERVED.